Aim: Write a program for frame sorting technique used in buffers.

Theory:

In computer networking, messages are broken down into discrete units called **frames** (Data Link Layer) or **packets/segments** (Network/Transport Layers).

Due to dynamic routing, variable queuing delays, and link jitter, frames often arrive at the receiver **out of sequence**. Before these frames can be passed up the protocol stack to the application layer, they must be temporarily held in a **receiver buffer** and reordered by their sequence numbers.\

Insertion Sort:

Insertion Sort builds the final sorted array one element at a time. It logically divides the buffer into two partitions:

1. **Sorted Subarray:** Frames that have already been placed in sequential order (indices 0 to $i - 1$).
2. **Unsorted Subarray:** Newly arrived or uninspected frames (indices i to $n - 1$).

Algorithmic Mechanics

For each step i from 1 to $n - 1$:

1. **Key Selection:** The frame at index i is designated as the key frame.
2. **Backward Scan & Right-Shift:** The algorithm scans backward through the sorted partition starting at index $j = i - 1$. For every frame whose sequence number is greater than key.seq_no, that frame is shifted one slot to the right (buffer[j + 1] = buffer[j]).
3. **Insertion:** When an element with a sequence number less than or equal to key.seq_no is encountered (or the beginning of the buffer is reached), the key frame is placed into the vacated position (buffer[j + 1] = key)

Insertion Sort is Well-Suited for Frame Buffers:

 Adaptive Speed: Runs in near-linear $O(N)$ time because network frames typically arrive with only minor order displacements.

 Online Execution: Inserts each incoming frame directly into its proper sorted slot the moment it hits the buffer without waiting for the whole batch.

 In-Place Operation: Uses strictly $O(1)$ auxiliary memory by rearranging frames directly inside the existing buffer array.

 Reduced Memory Writes: Moves heavy frame structures using a single right-shift per position rather than costly 3-step bubble swaps.

 Algorithmic Stability: Preserves the relative arrival order of frames bearing duplicate sequence numbers for easy duplicate detection.

 Minimal Implementation Overhead: Requires no recursion, extra allocation libraries, or complex data structures, making it ideal for low-level network drivers.

Source Code:

```
#include <stdio.h>
// Structure to store a frame
struct Frame {
   int seq_no;
   char data[50];
};

// Function to sort frames based on sequence numbers using Insertion Sort
void sortFrames(struct Frame buffer[], int n) {
   int i, j;
   struct Frame current_frame;

   for (i = 1; i < n; i++) {
      current_frame = buffer[i];
      j = i - 1;

      // Shift frames with a higher sequence number to the right
      while (j >= 0 && buffer[j].seq_no > current_frame.seq_no) {
         buffer[j + 1] = buffer[j];
         j--;
      }

      // Insert the frame in its correct position
      buffer[j + 1] = current_frame;
   }
}

int main() {
```

```
    int n, i;
    struct Frame buffer[50];
    // 1. Get total number of frames
    printf("Enter the number of frames: ");
    scanf("%d", &n);
    // 2. Read frames in random arrival order
    printf("\nEnter frames in random arrival order:\n");
    for (i = 0; i < n; i++) {
        printf("Frame %d -> Enter sequence number: ", i + 1);
        scanf("%d", &buffer[i].seq_no);
        printf("Frame %d -> Enter frame data: ", i + 1);
        scanf("%s", buffer[i].data);
    }
    // 3. Display frames as received
    printf("\n--- Frames Received in Buffer (Random Order) ---\n");
    for (i = 0; i < n; i++) {
        printf("Seq No: %d | Data: %s\n", buffer[i].seq_no, buffer[i].data);
    }
    // 4. Sort frames
    sortFrames(buffer, n);
    // 5. Display sorted frames
    printf("\n--- Frames After Sorting (Sequential Order) ---\n");
    for (i = 0; i < n; i++) {
        printf("Seq No: %d | Data: %s\n", buffer[i].seq_no, buffer[i].data);
    }
    return 0;
}
```

Output:

```
Enter frames in random arrival order:
Frame 1 -> Enter sequence number: 4
Frame 1 -> Enter frame data: CN
Frame 2 -> Enter sequence number: 1
Frame 2 -> Enter frame data: Hello
Frame 3 -> Enter sequence number: 5
Frame 3 -> Enter frame data: Lab
Frame 4 -> Enter sequence number: 2
Frame 4 -> Enter frame data: Welcome
Frame 5 -> Enter sequence number: 3
Frame 5 -> Enter frame data: to

--- Frames Received in Buffer (Random Order) ---
Seq No: 4 | Data: CN
Seq No: 1 | Data: Hello
Seq No: 5 | Data: Lab
Seq No: 2 | Data: Welcome
Seq No: 3 | Data: to

--- Frames After Sorting (Sequential Order) ---
Seq No: 1 | Data: Hello
Seq No: 2 | Data: Welcome
Seq No: 3 | Data: to
Seq No: 4 | Data: CN
Seq No: 5 | Data: Lab
```